

M2 명령줄 인터페이스 결과

2026. 10. 09.

목차

1. 요약
 2. 목표와 완료 기준
 3. 수행 내용
 4. 결과
 5. 논의
 6. 다음 단계
- 참고 출처

문서의 목적. M2 설계[D1]대로 만든 명령줄 인터페이스의 결과를 보고한다. M2가 마지막 마일스톤이므로 프로포절의 목표 전체 성공 기준도 함께 판정하고, M2와 이 목표를 끝내는 것의 승인을 받는다.

1 요약

`convert.py`에 `main()`을 더해 `python convert.py 100 km mi`가 `62.13711922 mi`를 출력한다. **M2 완료 기준 5개와 프로포절 성공 기준 4개를 모두 충족했다.**

- **시험.** 실제 프로세스를 띄우는 CLI 시험 10건과 M1 시험 127건, 모두 137건이 통과했다. 다른 폴더에서 실행해도 통과했다.
- **직접 실행에서 찾은 문제.** 숫자가 아닌 값을 넣으면 내부 함수 이름이 섞인 “invalid _finite value”가 나왔다. 시험은 “invalid”라는 글자만 확인해서 이것을 놓쳤다. 메시지를 “not a number: 'abc'”로 고치고 시험도 전체 문구를 확인하게 바꿨다.
- **남은 한계.** Python 3.11에서 `-1e3 m km`는 여전히 “not a number: 'm'”이라는 오해하기 쉬운 오류를 낸다. 설계대로 도움말에 -- 안내만 넣었다.

2 목표와 완료 기준

표 1은 M2 설계 표 5의 완료 기준과 판정이다[D1]. 표 2는 프로포절 표 4의 목표 전체 성공 기준과 판정이다[D3].

표 1. M2 완료 기준과 판정. [D1, D4]

기준	목표	결과	판정
정상 출력	3 / 3 (100 km mi, -40 C F, -- -1e3 m km)	3 / 3	충족
오류	6 / 6, 종료 2, 표준 출력 비어 있음	6 / 6	충족
도움말	1 / 1, 단위 목록과 -- 안내	1 / 1	충족
회귀	M1 시험 7 / 7	7 / 7	충족
의존성	argparse, math만	argparse, math	충족

표 2. 프로포절 성공 기준(목표 전체)과 판정. [D3, D4, D5]

기준	목표	결과	판정
정확성	상대 오차 $\leq 1e-12$	최대 $1.4e-15$ (M1)	충족
왕복 일관성	상대 오차 $\leq 1e-12$	109쌍 최대 $1.4e-16$ (M1)	충족
오류 처리	4 / 4가 메시지와 종료 코드 $\neq 0$	4종 + nan·inf 2종, 모두 종료 2와 이유	충족
의존성	표준 라이브러리만	argparse, math	충족

3 수행 내용

설계의 순서(시험 먼저 $\rightarrow$ main() $\rightarrow$ 전체 실행과 직접 확인)를 따랐다. 판정 어휘는 M1과 같이 채택 / 옵션 / 채택 안 함 / 보류다. 처음 시험을 돌렸을 때 CLI 하위 시험 10건이 실패했다. main()이 아직 없어서 생긴 의도한 실패였다[D4].

3.1 입력과 출력 (convert.py)

① 유한수만 받는 값 변환 (M2.1) — 채택

이유. float()은 "nan", "inf"를 숫자로 받아들여 변환까지 그대로 흘리간다(설계 표 2)[D1].

방식. argparse의 type에 _finite() 함수를 주었다. float()로 바꾼 뒤 math.isfinite()가 거짓이면 ArgumentTypeError를 낸다[R1, R2].

결과. nan km mi, inf km mi 모두 “value must be a finite number, got 'nan'” 형식의 오류와 종료 코드 2로 끝났다.

② 유효숫자 10자리 출력과 오류 통일 (M2.2) — 채택

이유. 부동소수점 잔여(211.99999999999994)를 사용자에게 보이지 않게 하고, 입력 해석 오류와 변환 오류가 같은 모양으로 끝나게 하려 했다[D1].

방식. 결과는 f"{result:.10g} {dst}"로 출력한다[R3]. convert()의 ValueError는 parser.error()로 넘겼다[R1]. 도움말 끝에는 M1의 KINDS 사전으로 만든 단위 목록과 -- 안내를 넣었다.

결과. 100 C F → 212 F로 나왔다. 오류 6종은 모두 사용법 한 줄과 convert.py: error: ... 한 줄, 종료 코드 2로 끝났다(표 3).

3.2 직접 확인에서 고친 것

③ 내부 함수 이름이 보이는 오류 메시지 (M2.3) — 채택

이유. 시험이 모두 통과한 뒤 셸에서 직접 실행해 보니, abc km mi가 “argument value: invalid _finite value: 'abc'”를 냈다. argparse는 type 함수가 ValueError를 내면 함수 이름을 메시지에 넣는다. 시험은 “invalid”라는 글자만 확인했기 때문에 이것을 놓쳤다.

방식. _finite() 안에서 float()의 ValueError를 잡아 ArgumentTypeError("not a number: 'abc'")로 바꿨다. 시험의 기대 문구도 “invalid”에서 “not a number: 'abc'”로 바뀌, 예전 메시지라면 실패하게 했다.

결과. 메시지는 “argument value: not a number: 'abc'”가 되었고, 시험 10개 메시드가 모두 다시 통과했다. 도움말의 value 인자에 설명(“number to convert”)도 함께 넣었다.

④ -1e3 오류 메시지 개선 (M2.4) — 보류

이유. Python 3.11에서 `-1e3 m km`는 `-1e3`을 옵션으로 보고 `m`을 값으로 읽는다[R1]. 그래서 지금은 “not a number: 'm'”이 나온다. 무엇이 잘못됐는지 알기 어렵다.

방식. 설계에서 정한 대로 도움말 끝에 “negative numbers in exponent form need '--' first” 안내만 넣었다. 오류 메시지 자체에 이 안내를 붙이는 방법(한 줄 추가)도 있지만, 설계 범위 밖이라 하지 않았다.

결과. `-- -1e3 m km` → `-1 km`는 정상이다. 메시지 개선은 6절에 선택 사항으로 남긴다. Python 3.14 이상에서는 이 문제가 없다[R1].

4 결과

표 3은 셸에서 직접 실행한 기록이다. 출력이 읽기 좋은지는 눈으로 판단했다. 그림 1은 M1 대비 코드 크기의 변화다.

표 3. 셸 실행 기록. Windows 11, Python 3.11.16, 2026.10.09. 오류는 마지막 줄만 적었다. 사용법 줄 `usage: convert.py [-h] value src dst`가 그 위에 붙는다. [D4]

입력	출력	종료
100 km mi	62.13711922 mi	0
100 C F	212 F	0
72 in cm	182.88 cm	0
1 lb oz	16 oz	0
1e-7 km mm	0.1 mm	0
-- -1e3 m km	-1 km	0
abc km mi	convert.py: error: argument value: not a number: 'abc'	2
nan km mi	... value must be a finite number, got 'nan'	2
1 km xx	... unknown unit 'xx' (known: mm, cm, m, km, ..., c, f, k)	2
1 KM KG	... cannot convert length to mass	2
-300 c f	... -300 c is below absolute zero	2
-1e3 m km	... not a number: 'm' (3.2절 ④)	2

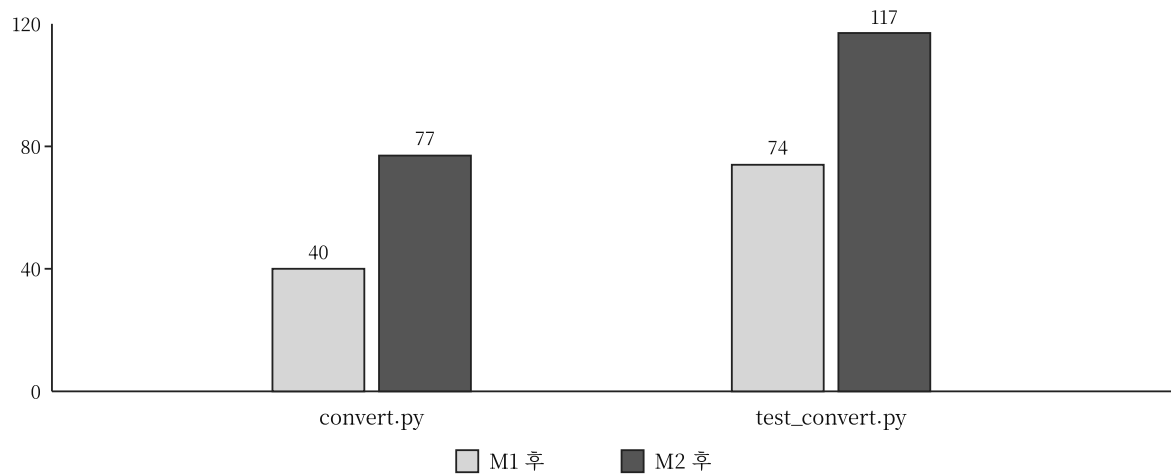


그림 1. 파일별 줄 수(M1 후와 M2 후). convert.py는 37줄 늘었고, 그중 변환 계산은 바뀌지 않았다. 늘어난 것은 입력 해석·출력·도움말이다. 시험 파일은 CLI 시험 43줄이 늘었다.

표 4. 측정값. [D4]

항목	측정	값
시험	메서드 수 / 확인 건수(M1 127 + CLI 10)	10 / 137
시험 실행 시간	python -m unittest, 프로세스 실행 10회 포함	0.65 s
CLI 1회 실행 시간	20회 중앙값 / 최대	61.3 / 65.5 ms
다른 폴더에서 시험	unittest discover -s <저장소>	통과

5 논의

통과하는 시험과 직접 실행은 서로 다른 것을 확인한다. 3.2절 ③의 메시지 문제는 시험이 “invalid”라는 글자만 확인해서 생겼다. 셸에서 한 번 실행해 보고서야 드러났다. 고친 뒤에는 오류 시험 6건이 모두 이유 문구 전체를 확인한다.

출력 형식이 M1의 부동소수점 잔여를 감춘다. M1에서 $1.4e-15$ 오차를 남긴 $-40\text{ }^{\circ}\text{C} \rightarrow\text{ }^{\circ}\text{F}$ 는 화면에 -40으로 나온다[D2]. 계산은 바뀌지 않았고 보이는 자릿수만 10자리로 제한했다.

CLI 1회 실행 시간 61 ms는 대부분 Python 시작 시간으로 보인다(검증 안 함). M1에서 켜 변환 1회는 1 μs 미만이었다.

증거의 한계도 있다. 시험은 Windows 11의 Python 3.11.16에서만 돌렸다. 다른 운영체제와 Python 3.14 이상에서의 동작, 그리고 Windows 콘솔 인코딩 영향은 확인하지 않았다(검증 안 함). 출력이 ASCII뿐이라 영향은 없을 것으로 본다.

6 다음 단계

이 목표의 마일스톤은 모두 끝났다. 표 5는 남은 선택 사항이다. 코드와 문서는 아직 커밋하지 않았다 [D4].

표 5. 다음 단계.

작업	내용	필요한 것
커밋	convert.py, test_convert.py, docs/reports/ 커밋 (지금 브랜치는 master)	개발자 요청
-le3 메시지 (옵션)	“not a number” 오류에 -- 안내를 한 줄 덧붙이기	개발자 결정
온도 차·복합 단위	프로포절에서 범위 밖으로 둔 확장	새 목표로 시작

참고 출처

- [R1] Python Software Foundation. argparse — Parser for command-line options, arguments and subcommands. <https://docs.python.org/3/library/argparse.html>
- [R2] Python Software Foundation. math.isfinite. <https://docs.python.org/3/library/math.html#math.isfinite>
- [R3] Python Software Foundation. Format Specification Mini-Language. <https://docs.python.org/3/library/string.html#format-specification-mini-language>
- [D1] M2 명령줄 인터페이스 설계 (docs/reports/unit-converter-cli/M2-design.pdf), 2026.10.09 승인.
- [D2] M1 변환 핵심 결과 (docs/reports/unit-converter-cli/M1-result.pdf), 2026.10.09 승인.
- [D3] 단위 변환 CLI 프로포절 (docs/reports/unit-converter-cli/00-proposal.pdf), 2026.10.09 승인.
- [D4] convert.py, test_convert.py와 실행·측정 기록, Python 3.11.16, 2026.10.09 (커밋 전).
- [D5] M1 결과의 오차 측정 (D2의 표 2).